

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
AI

Practical – VI

Roll No.: T023	Name: Som Mishra
Class: TYIT	Batch: 1
Date of Assignment: 19/8/26	Date/Time of Submission: 19/8/26

Probability Reasoning (Bayesian Networks & Inference)

- Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code :

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

✓ 0.0s

Step 1: Define Network Structure (using DiscreteBayesianNetwork) #

```
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
```

Step 2: Define Conditional Probability Tables (CPDs)

P(Burglary)

```

cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],

    state_names={"Burglary": ["False", "True"]}
)

```

```

P(Earthquake) cpd_earthquake
= TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)

```

```

P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)

```

```

P(JohnCalls | Alarm) cpd_john
= TabularCPD(

```

```

variable="JohnCalls",
variable_card=2,
values=[
    [0.95, 0.10], # JohnCalls = False
    [0.05, 0.90] # JohnCalls = True
],
evidence=["Alarm"],
evidence_card=[2],

state_names={
    "JohnCalls": ["False", "True"],
    "Alarm": ["False", "True"]
}
)

```

```

P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30], # MaryCalls = False
        [0.01, 0.70] # MaryCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "MaryCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)

```

Step 3: Add CPDs to the Model and Validate #

```
model.add_cpds(cpd_burglary, cpd_earthquake, cpd_alarm, cpd_john, cpd_mary)
```

Verify valid CPDs and correct total probability assert model.check_model(),
"Model check failed!"

Step 4: Perform Variable Elimination Inference #
infer = VariableElimination(model)

Query: $P(\text{Burglary} \mid \text{JohnCalls}=\text{True}, \text{MaryCalls}=\text{True})$
query_result = infer.query(
 variables=["Burglary"],

 evidence={"JohnCalls": "True", "MaryCalls": "True"}
)

print(query_result)

Output:

Burglary	phi(Burglary)
Burglary(False)	0.7158
Burglary(True)	0.2842

- Write a program to compute Posterior Probability using Bayesian Networks(Burglar|Alarm = True)

Code :

Step 1: Create Bayesian Network structure

```
model = DiscreteBayesianNetwork([  
    ("Burglary", "Alarm"),  
    ("Earthquake", "Alarm")  
])
```

Step 2: Define probability of Burglary

```
cpd_burglary = TabularCPD(  
    variable="Burglary",  
    variable_card=2,  
    values=[[0.999], [0.001]],  
    state_names={"Burglary": ["False", "True"]}  
)
```

Step 3: Define probability of Earthquake

```
cpd_earthquake = TabularCPD(  
    variable="Earthquake",  
    variable_card=2,  
  
    values=[[0.998], [0.002]],  
    state_names={"Earthquake": ["False", "True"]}  
)
```

Step 4: Define probability of Alarm $P(\text{Alarm} \mid \text{Burglary}, \text{Earthquake})$ cpd_alarm

```
= TabularCPD(  
    variable="Alarm",  
    variable_card=2,  
    values=[  
        [0.999, 0.71, 0.06, 0.05], # Alarm = False  
        [0.001, 0.29, 0.94, 0.95] # Alarm = True  
    ],  
    evidence=["Burglary", "Earthquake"],  
    evidence_card=[2, 2],  
    state_names={  
        "Alarm": ["False", "True"],  
        "Burglary": ["False", "True"],  
        "Earthquake": ["False", "True"]  
    }  
)
```

Step 5: Add CPDs to model `model.add_cpds(cpd_burglary, cpd_earthquake, cpd_alarm)`

Step 6: Check whether model is correct

```
print("Is Bayesian Network valid?", model.check_model())
```

Step 7: Perform inference

```
inference = VariableElimination(model)
```

```
result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)
```

```
print("\nPosterior Probability:")
print("P(Burglary | Alarm = True)")
print(result)
```

Output :

```
Is Bayesian Network valid? True

Posterior Probability:
P(Burglary | Alarm = True)
+-----+-----+
| Burglary | phi(Burglary) |
+=====+=====+
| Burglary(False) | 0.6264 |
+-----+-----+
| Burglary(True) | 0.3736 |
+-----+-----+
```

Extra :

Machine Failure Prediction:

Overheating → machine temperature is high
High Vibration → abnormal vibration detected
Machine Failure → machine fails

Code :

```
1. Create Bayesian Network model
= DiscreteBayesianNetwork([
    ('Overheating', 'Machine_Failure'),
    ('High_Vibration', 'Machine_Failure')
])
```

```
2. Define CPD for Overheating
cpd_overheating = TabularCPD(
    variable='Overheating',
    variable_card=2,
    values=[
        [0.70], # False
        [0.30] # True
    ]
)
```

```
3. Define CPD for High Vibration
cpd_vibration = TabularCPD(
    variable='High_Vibration',
    variable_card=2,
    values=[
        [0.60], # False
        [0.40] # True
    ]
)
```

```
4. Define CPD for Machine Failure
Evidence: Overheating, High_Vibration
cpd_failure = TabularCPD(
    variable='Machine_Failure',
    variable_card=2,
```

```

values=[
    [0.99, 0.80, 0.70, 0.20], # Failure = False
    [0.01, 0.20, 0.30, 0.80]  # Failure = True
],
evidence=['Overheating', 'High_Vibration'],
evidence_card=[2, 2]
)

```

5. Add CPDs to the model

```

model.add_cpds(
    cpd_overheating,
    cpd_vibration,
    cpd_failure
)

```

6. Check whether the model is valid

```

print("Model is valid:", model.check_model())

```

7. Perform Bayesian Inference

```

inference = VariableElimination(model)

```

8. Calculate $P(\text{Machine_Failure}=\text{True} \mid$

$\text{Overheating}=\text{True}, \text{High_Vibration}=\text{True})$

```

result = inference.query(
    variables=['Machine_Failure'],
    evidence={
        'Overheating': 1,
        'High_Vibration': 1

```

```

    }

```

```

)

```

```

print("\nPosterior Probability:")

```

```

print(result)

```



```
# 9. Display probability of machine failure
failure_probability = result.values[1]
```

```
print(
    "\nProbability of Machine Failure =",
    round(failure_probability * 100, 2),
    "%"
)
```

```
... Model is valid: True

Posterior Probability:
+-----+
| Machine_Failure | phi(Machine_Failure) |
+=====+
| Machine_Failure(0) | 0.2000 |
+-----+
| Machine_Failure(1) | 0.8000 |
+-----+

Probability of Machine Failure = 80.0 %
```

Output :